

UNIVERSIDAD CARLOS III



Procesadores del lenguaje

Práctica Final:

Traductor frontend: subconjunto de lenguaje C a LISP (código intermedio)

Traductor backend: LISP a notación Postfija (código final)

Grupo 503

CURSO 2025 - 2026

RAQUEL DEL VALLE OLIVARES → 100522214@alumnos.uc3m.es

Grupo 85, Ingeniería Informática

XIYA YE → 100522159@alumnos.uc3m.es

Grupo 85, Ingeniería Informática

ÍNDICE

FRONTEND - Especificación 1: Variables Globales.....	3
FRONTEND - Especificación 2: Función MAIN.....	3
FRONTEND - Especificación 3: Impresión de cadenas.....	4
FRONTEND - Especificación 4: Impresión expresiones y cadenas.....	5
FRONTEND - Especificación 5: Operadores, precedencia y asociatividad.....	6
FRONTEND - Especificación 6: Estructura de control WHILE.....	7
FRONTEND - Especificación 7: Estructura de control IF.....	8
FRONTEND - Especificación 8: Variables Locales.....	9
FRONTEND - Especificación 9: Estructura de control FOR.....	10
FRONTEND - Especificación 10: Estructura de control Switch/Case.....	11
FRONTEND - Especificación 11: Funciones.....	13
11.a. Función sin valor de retorno, tipo explícito ni parámetros.....	13
11.b. Función con parámetros.....	14
11.c. Retorno de funciones.....	15
11.d. Llamadas a funciones como procedimientos.....	17
FRONTEND - Especificación 12: Implementación de Vectores.....	17
12.a. Declaración de vectores.....	17
12.b. Acceso a elementos de un vector (operando).....	18
12.c. Modificación de elementos de un vector (receptor).....	18
12.d. Vectores como globales, locales y argumentos.....	19
BACKEND - Especificación 1: Variables Globales.....	20
BACKEND - Especificación 2: Función MAIN.....	20
BACKEND - Especificación 4: Impresión de enteros.....	21
BACKEND - Especificación 5: Asignación.....	21
BACKEND - Especificación 6: Estructura de control WHILE.....	21
BACKEND - Especificación 7: Estructura de control IF.....	22
BACKEND - Especificación 8: Operadores.....	22

El trabajo ha sido realizado por ambas integrantes en partes iguales. Se ha ido realizando en las sesiones semanales y si no nos daba tiempo a acabar, íbamos a la biblioteca para realizarlo juntas.

No se ha usado la IA generativa ni para el código ni para la memoria.

FRONTEND - Especificación 1: Variables Globales

Para que nuestro traductor lea las declaraciones de las variables globales, hemos empezado por crear una nueva regla (*declaracion_vars*) que reconozca el token *INTEGER* (correspondiente a la palabra reservada *int*) seguido de una lista de variables y terminado con un punto y coma.

A continuación, implementamos otra regla *listas* de forma recursiva por la izquierda. Para que nos permita procesar secuencias de variables separadas por comas. Las producciones para esta regla son:

- IDENTIF
Que nos traduce asignando el valor por defecto, 0.
- IDENTIF '=' NUMBER
Se nos traduce asignando el valor constante introducido.
- lista ',' IDENTIF y lista ',' IDENTIF '=' NUMBER
Estos nos permiten concatenar las traducciones previas de la lista con la nueva variable.

PRUEBAS:

Para comprobar el funcionamiento hemos realizado las siguientes pruebas:

- | | |
|------------------|----------------------------------|
| - Entrada: | Salida: |
| int a; | (setq a 0) |
| int a=3, b=1, c; | (setq a 3) (setq b 1) (setq c 0) |

FRONTEND - Especificación 2: Función MAIN

Para que nuestro traductor pueda leer la función main, hemos empezado por modificar la regla principal *axioma* para forzar que la estructura del programa C siga un orden lógico, siendo primero las declaraciones de variables globales y después la función main.

Siguiendo las especificaciones pedidas, aislamos las sentencias ejecutables de manera exclusiva. Además, implementamos de forma recursiva por la izquierda una nueva regla *lista_sentencias*. Este nos permite agrupar varias instrucciones terminadas por punto y coma dentro del bloque delimitado por las llaves *{ }*.

Finalmente, creamos la función *funcion_main* que reconoce la estructura del main en lenguaje C. Su traducción a Lisp consiste en envolver el código generado por las sentencias dentro de (*defun main()*...) y a continuación, añadir una llamada a (*main*) fuera de la definición para la llamada a ejecutar el programa.

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	Salida:
<pre>main () { b = a + 2; @ b; } //@ (main)</pre>	<pre>(defun main () (setq b (+ a 2)) (print b)) (main)</pre>
- Entrada:	Salida:
<pre>main () { @ (a+1); } //@ (main)</pre>	<pre>(defun main () (print (+ a 1))) (main)</pre>

FRONTEND - Especificación 3: Impresión de cadenas

Para sustituir el uso temporal del carácter @ en la impresión de mensajes, hemos implementado el soporte para la función PUTS.

Primero, hemos actualizado el analizador léxico añadiendo la palabra reservada “puts” a nuestra tabla de símbolos (*keywords*) y declaramos el nuevo token *%token PUTS* en la cabecera.

A continuación introducimos una nueva regla dentro de sentencia:

```
C/C++  
| PUTS '(' STRING ')'  
{ sprintf (temp, "(print \"%s\\")", $3.code) ;  
  $$code = gen_code (temp) ; }
```

Le añadimos unas comillas al print ya que hemos detectado que al guardar las cadenas en \$3.code , les quita las comillas (ya que la primera no la guarda y la última la intercambia por el final de cadena \0). Es decir, al escribir “Hola mundo” , el léxico guarda Hola mundo.

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	Salida:
<pre>int a; main () { puts(“Hola mundo”); } //@ (main)</pre>	<pre>(setq a 0) (defun main () (print “Hola mundo”)) (main)</pre>

FRONTEND - Especificación 4: Impresión expresiones y cadenas

Para este apartado, hemos ampliado la gramática para reconocer la sentencia *printf* con un número variable de parámetros: *printf(<string_formato> , <elem1> , ... , <elemN>);*

Para ello, añadimos el token *PRINTF* y su correspondiente entrada en la tabla de palabras reservadas (igual que con *puts*). A continuación se muestra como lo hemos implementado en la gramática:

- Regla dentro de *sentencia*: Siguiendo las especificaciones del enunciado, el primer argumento de un *printf* en C es la cadena de formato (\$3). Dado que solo se pide su reconocimiento por la gramática y no su traducción, lo ignoramos intencionadamente en la acción semántica, subiendo únicamente el código generado por el resto de argumentos (\$\$.code = \$4.code).

C/C++

```
| PRINTF '(' STRING lista_printf ')' { $$code = $4.code ; }
```

- Lista variable de argumentos (*lista_printf* y *elem_printf*): Para soportar infinitos argumentos separados por comas, hacemos uso de la recursividad por la izquierda en la regla *lista_printf*. Cada elemento individual puede ser evaluado como una *expresion* o como un *STRING*. En ambos casos se traducen con la función *princ* en lugar de *print*. Esto se debe a que *princ* omite el salto de línea final permitiendo que todos los elementos del *printf* se impriman de forma continua (sin saltos de línea) en la consola. Al igual que en *PUTS*, si el elemento es un *STRING*, le restauramos las comillas en la traducción.

C/C++

```
lista_printf:  ',' elem_printf          { $$code = $2.code; }
              | lista_printf ',' elem_printf { sprintf (temp, "%s\n%s",
                                              $1.code, $3.code) ;
                                              $$code = gen_code (temp) ; }
;

elem_printf:  expresion                { sprintf (temp, "(princ %s)", $1.code) ;
                                         $$code = gen_code (temp) ; }
              | STRING                 { sprintf (temp, "(princ \"%s\")", $1.code) ;
                                         $$code = gen_code (temp) ; } ;
```

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	Salida:
<code>int edad;</code>	<code>(setq edad 0)</code>
<code>main () {</code>	<code>(defun main ()</code>
<code> edad = 20;</code>	<code>(setq edad 20)</code>
<code> printf("txt ignorado", "Tienes ",</code>	<code>(princ "Tienes ")</code>
<code> edad, " años y el doble es ",</code>	<code>(princ edad)</code>
<code> edad * 2);</code>	<code>(princ " años y el doble es ")</code>
<code>}</code>	<code>(princ (edad * 2))</code>
	<code>)</code>
<code>//@ (main)</code>	<code>(main)</code>

FRONTEND - Especificación 5: Operadores, precedencia y asociatividad

En este apartado hemos ampliado la gramática para soportar los operadores lógicos (`&&`, `||`, `!`) y relacionales (`==`, `!=`, `<`, `>`, `<=`, `>=`), así como el operador aritmético módulo (`%`).

Como en los pasos anteriores, primero actualizamos el analizador léxico añadiendo los operadores compuestos (los de más de 1 carácter) a la tabla de palabras reservadas (*keywords*). A continuación, declaramos sus respectivos tokens (*AND*, *OR*, *EQ*, *NEQ*, *LEQ*, *GEQ*) en la cabecera.

Para garantizar que se respete la jerarquía matemática del lenguaje C y evitar conflictos de derivación (Shift/Reduce) al evaluar expresiones complejas, configuramos las directivas de precedencia de Bison según la tabla del link proporcionado en el enunciado ([Link C Operator Precedence](#)) ordenadas de menor a mayor prioridad. Finalmente, añadimos las nuevas producciones a las reglas *expresion* y *termino* traduciendo cada operador a su equivalente en Lisp (por ejemplo, de `&&` a *and*).

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	
<code>int a;</code>	
<code>main () {</code>	
<code> a = 10;</code>	
<code> printf("Test lógico: ", a > 5 && !(a == 20) a % 2 != 0);</code>	
<code>}</code>	
- Salida	
<code>(setq a 0)</code>	
<code>(defun main ()</code>	
<code>(setq a 10)</code>	
<code>(princ (or (and (> a 5) (not (= a 20))) (/= (mod a 2) 0)))</code>	
<code>)</code>	
<code>(main)</code>	

FRONTEND - Especificación 6: Estructura de control WHILE

Para implementar el bucle *while*, registramos primero la palabra reservada en la tabla *keywords* asociándola al token *WHILE*.

Para el manejo del punto y coma (;) hemos eliminado el requerimiento del ; en *lista_sentencias* y lo trasladamos individualmente al final de las producciones que lo requieren explícitamente (todas menos el *while*).

C/C++

```
| WHILE '(' expresion ')' '{' lista_sentencias '}' {  
    sprintf (temp, "(loop while %s do\n%s)", $3.code, $6.code) ;  
    $$code = gen_code (temp) ; }
```

Respecto a lo de evaluar aritméticas como booleanos (ej. *while(10-a)*), de momento realizamos una traducción literal a (*loop while (- 10 a) do ...*), dejando para más adelante la resolución de la misma para fases posteriores si así lo indicara el enunciado.

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	Salida:
<code>int a;</code>	<code>(setq a 0)</code>
<code>main () {</code>	<code>(defun main ()</code>
<code> a = 0;</code>	<code>(setq a 0)</code>
<code> while(a < 5) {</code>	<code>(loop while (< a 5) do</code>
<code> printf("Ignorado", a);</code>	<code>(princ a)</code>
<code> a = a + 1;</code>	<code>(setq a (+ a 1)))</code>
<code> }</code>	
<code>}</code>	<code>)</code>
<code>//@ (main)</code>	<code>(main)</code>

FRONTEND - Especificación 7: Estructura de control IF

Para este apartado, ampliamos la gramática para dar soporte a las construcciones condicionales *if-then* e *if-then-else*. Comenzamos registrando los tokens *IF* y *ELSE* en el analizador léxico.

Al diseñar la gramática, observamos que ocurre el problema de ambigüedad Dangling Else. Si definiéramos las reglas condicionales sin delimitadores explícitos, un parser Bottom-Up arrojaría un conflicto Shift/Reduce al no poder determinar exactamente a qué *if* pertenece un *else* anidado. Para resolver este problema, hemos implementado el uso de marcadores de bloque (*{}*) obligatorios para todas las ramas condicionales. Gracias a esto, eliminamos la ambigüedad.

Por otro lado, en C, las llaves permiten agrupar un número indeterminado de instrucciones pero en Lisp, la función *if* solo acepta una única forma/expresión para su rama verdadera y patria para la falsa. Por tanto, para garantizar una traducción robusta de las instrucciones dentro de las llaves, utilizamos la función *progn* de Lisp. Esta función evalúa secuencialmente una lista de expresiones, actuando como el equivalente semántico del bloque *{}* en C.

C/C++

```
| IF '(' expresion ')' '{' lista_sentencias '}'          {
|     sprintf (temp, "(if %s\n (progn %s)\n)", $3.code, $6.code) ;
|     $$code = gen_code (temp) ; }
| IF '(' expresion ')' '{' lista_sentencias '}' ELSE '{' lista_sentencias '}' {
|     sprintf (temp, "(if %s\n (progn %s)\n (progn %s)\n)", $3.code, $6.code,
|         $10.code) ;
|     $$code = gen_code (temp) ; }
```

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada: <pre>int a = 1; int b; main() { if (a == 0){ b = 123; } else { b = 456; } printf("%d", b); } //@ (main)</pre>	Salida: <pre>(setq a 0) (setq b 0) (defun main () (if (= a 0) (progn (setq b 123)) (progn (setq b 456)))) (princ b)) (main)</pre>
---	--

FRONTEND - Especificación 8: Variables Locales

Para esta especificación, hemos tratado el soporte para variables locales dentro de las funciones y la resolución de colisiones de nombres (una variable local que se llame igual que una global).

Como Lisp trata las variables creadas con *setq* como globales por defecto, simularemos el ámbito local prefijando el nombre de la variable con el nombre de la función (ej. *main_a*). Para conseguirlo de forma dinámica, implementamos una Tabla de Símbolos Local. Se crea una estructura de array *local_vars* y funciones auxiliares (*add_local_var*; *is_local_var*). Además, desarrollamos la función *get_var_name* que actúa como interceptor: recibe el identificador de una variable detectada por Bison y comprueba la tabla; si la variable es local, devuelve el string modificado con el prefijo *main_*; si no lo es, asume que es una variable global y lo devuelve sin ningún cambio.

A nivel sintáctico hemos hecho los siguientes cambios:

- Añadimos las reglas *declaraciones_locales* y *lista_local* al inicio de la función *main* para soportar las declaraciones internas en C e ir añadiendo las variables a nuestra tabla de símbolos durante el análisis. En estas declaraciones seguimos usando *setq*.

C/C++

```
MAIN '(' ')' '{' declaraciones_locales lista_sentencias '}'
    { sprintf (temp, "(defun main ()\n%s\n%s\n)\n(main)", $5.code, $6.code) ;
      $$code = gen_code (temp) ; }
```

- Actualizamos las traducciones en la regla *sentencia* (para las asignaciones) y en la regla *operando* envolviendo los identificadores con la función *get_var_name*.

C/C++

```
sentencia:      IDENTIF '=' expresion ';'      { sprintf (temp, "(setf %s %s)",
                                                get_var_name($1.code), $3.code) ;
                                                $$code = gen_code (temp) ; }
```

```
operando:      IDENTIF      { sprintf (temp, "%s", get_var_name($1.code)) ;
                             $$code = gen_code (temp) ; }
```

- Cambiamos la traducción de la sentencia de asignación para que haga uso de *setf*, diferenciando semánticamente una declaración de una asignación en Lisp.

C/C++

```
sentencia:      IDENTIF '=' expresion ';'      { sprintf (temp, "(setf %s %s)",
                                                get_var_name($1.code), $3.code) ;
                                                $$code = gen_code (temp) ; }
```

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	Salida:
<code>int a = 1;</code>	<code>(setq a 0)</code>
<code>main () {</code>	<code>(defun main ()</code>
<code>int a = 4;</code>	<code>(setq main_a 4)</code>
<code>a = a + 1;</code>	<code>(setf main_a (+ main_a 1))</code>
<code>printf("%d", a+1);</code>	<code>(princ (+ main_a 1))</code>
<code>}</code>	<code>)</code>
<code>//@ (main)</code>	<code>(main)</code>

FRONTEND - Especificación 9: Estructura de control FOR

Para este apartado, hemos empezado actualizando nuestro analizador léxico añadiendo las palabras reservadas *for*, *inc* y *dec* a la tabla *keywords*, asociándolas a sus tokens declaradas en la cabecera. Además, hemos tenido en cuenta que registramos *inc* y *dec* en minúsculas en la tabla, ya que nuestro analizador convierte todo el texto a minúsculas antes de buscar coincidencias, evitando así errores de sintaxis al procesar el código.

A nivel sintáctico, hemos añadido dos nuevas producciones a la regla *sentencia* (una para el incremento y otra para el decremento). La lógica de traducción que hemos aplicado es la siguiente:

1. La inicialización se extrae y se coloca justo antes de entrar al bucle mediante *setf*.
2. Evaluamos la condición en la cabecera del *loop while*.
3. Colocamos el cuerpo de bucle dentro del bucle *do*.
4. El incremento y decremento se traduce como una asignación +1 o -1 y lo añadimos al final del bloque de sentencias.

Además, en todas las partes donde interviene la variable índice, hacemos uso de la función *get_var_name()* para asegurarnos que se aplique correctamente el prefijo *main_* si se trata de una variable local.

C/C++

```
sentencia:  FOR '(' IDENTIF '=' expresion ';' expresion ';' INC '(' IDENTIF
            ')' ')' '{' lista_sentencias '}'      {sprintf (temp, "(setf %s
            %s)\n(loop while %s do\n%s\n(setf %s (+ %s 1)))",
            get_var_name($3.code), $5.code, $7.code, $15.code,
            get_var_name($11.code), get_var_name($11.code));
            $$code = gen_code (temp); }

sentencia:  FOR '(' IDENTIF '=' expresion ';' expresion ';' DEC '(' IDENTIF
            ')' ')' '{' lista_sentencias '}'      {sprintf (temp, "(setf %s
            %s)\n(loop while %s do\n%s\n(setf %s (- %s 1)))",
            get_var_name($3.code), $5.code, $7.code, $15.code,
            get_var_name($11.code), get_var_name($11.code));
            $$code = gen_code (temp); }
```

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	Salida:
<code>int a;</code>	<code>(setq a 0)</code>
<code>main() {</code>	<code>(defun main ()</code>
<code>int a = 4;</code>	<code>(setq main_a 4)</code>
<code>int i = 0;</code>	<code>(setq main_i 0)</code>
 <code>puts("Inicio del FOR:");</code>	 <code>(print "Inicio del FOR:");</code>
 <code>for (i = 0; i < 3; INC(i)) {</code>	 <code>(setf main_i 0)</code>
<code>a = a + 1;</code>	<code>(loop while (< main_i 3) do</code>
<code>printf("Valor de a: ", a);</code>	<code>(setf main_a (+ main_a 1))</code>
<code>puts("");</code>	<code>(princ main_a)</code>
<code>}</code>	<code>(print "")</code>
<code>}</code>	<code>(setf main_i (+ main_i 1)))</code>
<code>//@ (main)</code>	<code>)</code>
	<code>(main)</code>

FRONTEND - Especificación 10: Estructura de control Switch/Case

Para implementar la estructura *switch/case*, comenzamos registrando los nuevos tokens (*SWITCH*, *CASE*, *DEFAULT* Y *BREAK*) en el analizador léxico y añadiendo sus respectivas cadenas en minúsculas a la tabla de palabras reservadas *keywords*.

A nivel sintáctico, hemos definido una nueva regla *lista_casos* con recursividad por la izquierda, lo que nos permite evaluar un número indeterminado de casos. Donde cada caso individual se gestiona mediante la regla *caso*, que asocia un valor numérico (*NUMBER*) a un bloque de instrucciones (*lista_secuencias*). Además, para garantizar que Lisp evalúe correctamente estas sentencias múltiples dentro de un mismo caso, las hemos envuelto en una función *progn*, al igual que con la estructura de control IF.

C/C++

```
lista_casos:  caso          { $$ = $1; }
             | lista_casos caso { sprintf(temp, "%s\n%s", $1.code, $2.code);
                               $$code =gen_code(temp); }

caso:  CASE NUMBER ':' lista_sentencias BREAK ';'
      { sprintf(temp, "(%d(progn %s))", $2.value, $4.code);
        $$code =gen_code(temp); }
```

Finalmente, integramos la estructura global dentro de la regla *sentencia* añadiendo dos producciones: una para un *switch* estándar y otra para un *switch* que incluya un bloque *default*, ya que esto no lo podemos añadir en la *lista_casos*, y al ser un caso en el que a veces se puede no incluir. Además, en lugar de limitarnos a evaluar un identificador simple, hemos configurado el *switch* para que acepte cualquier *expresion*, aprovechando que dicha regla ya gestiona de forma transparente el mapeo de variables globales y locales.

C/C++

```

sentencia: SWITCH '(' IDENTIF ')' '{' lista_casos '}'
{ sprintf(temp, "(case %s\n%s)", get_var_name($3.code), $6.code);
  $$code = gen_code (temp) ; }

| SWITCH '(' IDENTIF ')' '{' lista_casos DEFAULT ':'
lista_sentencias BREAK ';' '}'
{ sprintf(temp, "(case %s\n%s\n(otherwise (progn %s)))",
  get_var_name($3.code), $6.code, $9.code);
  $$code = gen_code (temp) ; }

```

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

Entrada:	Salida:
<code>int estado;</code>	<code>(setq estado 0)</code>
<code>main() {</code>	<code>(defun main ()</code>
<code>int estado = 2;</code>	<code>(setq main_estado 2)</code>
<code>int valor = 0;</code>	<code>(setq main_valor 0)</code>
<code>switch (estado) {</code>	<code>(case main_estado</code>
<code>case 1:</code>	<code>(1(progn (setf main_valor 10)))</code>
<code>valor = 10;</code>	
<code>break;</code>	
<code>case 2:</code>	<code>(2(progn (setf main_valor 20)))</code>
<code>valor = 20;</code>	
<code>break;</code>	
<code>default:</code>	<code>(otherwise (progn (setf main_valor (- 1))))</code>
<code>valor = -1;</code>	
<code>break;</code>	
<code>}</code>	
<code>}</code>	<code>)</code>
<code>//@(main)</code>	<code>(main)</code>

FRONTEND - Especificación 11: Funciones

Para implementar esta parte, hemos ido abordando el problema de forma incremental.

11.a. Función sin valor de retorno, tipo explícito ni parámetros

En esta primera fase, hemos habilitado la creación y llamada a funciones básicas. Para ello, hemos modificado nuestro código C añadiendo una variable global *funcion_actual* y una función *limpia_vars_locales* para inicializar el número de variables locales cada vez que se llame a otra función. Hemos modificado la función interceptora *get_var_name* para que concatena dinámicamente el nombre de la función actual.

Para garantizar que el parser actualice el nombre de la función antes de leer su bloque de instrucciones (y evitar que las acciones semánticas desplacen los índices *\$1*, *\$2*, ...), hemos introducido unas reglas intermedias de cabecera:

```
C/C++
cabecera_funcion:
    IDENTIF '(' ')'          { strcpy(funcion_actual, $1.code);
                             limpia_vars_locales();
                             $$code = $1.code; }
    ;

cabecera_main:
    MAIN '(' ')'            { strcpy(funcion_actual, "main");
                             limpia_vars_locales(); }
    ;
```

Finalmente, hemos amplificado las reglas *sentencia* y *operando* para reconocer las llamadas a estas funciones sin parámetros.

```
C/C++
sentencia:  IDENTIF '(' ')' ';' { sprintf(temp, "(%s)", $1.code);
                                $$code = gen_code (temp); }
operando:  IDENTIF '(' ')'      { sprintf(temp, "(%s)", $1.code);
                                $$code = gen_code (temp); }
```

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	Salida:
<pre>funcion () { puts("print desde funcion"); }</pre>	<pre>(defun funcion () (print "print desde funcion"))</pre>
<pre>main () { funcion(); puts("print desde main"); } //@ (main)</pre>	<pre>(defun main () (funcion) (print "print desde main")) (main)</pre>

11.b. Función con parámetros

Para ampliar el traductor y permitir que las funciones acepten parámetros, hemos introducido una regla intermedia *nombre_funcion* que fija *funcion_actual* y limpia las tablas antes de que Bison procese los parámetros. Sin ella, cuando se redujera *lista_param_def*, *funcion_actual* aún tendría el nombre de la función anterior.

Para evitar que las acciones semánticas de la cabecera desplacen los índices de los símbolos, hemos integrado todos los componentes directamente en la regla *funcion*. De este modo, cada símbolo es accesible por su posición utilizando únicamente el campo *.code*.

```
C/C++
nombre_funcion:
    IDENTIF      { strcpy(funcion_actual, $1.code); limpia_vars_locales();
                  limpia_params();
                  $$code = $1.code; }
    ;

funcion:
    nombre_funcion '(' parametros_def ')' '{' declaraciones_locales lista_sentencias '}'
    { sprintf (temp, "(defun %s (%s)%s\n%s\n)", $1.code, $3.code,
      $6.code, $7.code) ;
      printf("%s\n", temp) ;
      $$code = gen_code (temp) ; }
    ;
```

Las reglas *parametros_def* y *lista_param_def* siguen el mismo patrón de procedimiento de contenedor con lambda más recursividad por la izquierda que el resto de listas del proyecto. En *lista_param_def* usamos *add_param* en lugar de *add_local_var* porque en Lisp los parámetros formales ya son locales por naturaleza. Para ello implementamos una tabla de parámetros independientemente con las funciones auxiliares *add_param*, *is_param* y *limpia_params*, y actualizamos *get_var_name* para consultarla. Para las llamadas, añadimos *parametros_llamada* y *lista_param_llamada* con la misma estructura, usando

expresion para permitir pasar no solo variables sino operaciones como $g+1$, y actualizamos *sentencia* y *operando* en consecuencia.

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada: <pre>int g = 10; suma (int x, int y) { printf("ignorado", x + y + g); } main () { suma(3, g+1); } //@ (main)</pre>	Salida: <pre>(setq g 10) (defun suma (x y) (princ (+ (+ x y) g))) (defun main () (suma 3 (+ g 1))) (main)</pre>
--	--

11.c. Retorno de funciones

Para implementar el retorno, distinguimos 2 casos según la posición del *RETURN* dentro de la función.

El primer caso corresponde a un *return* bien estructurado como última sentencia. En Lisp, el valor de retorno de una función es simplemente la última expresión evaluada, por lo que la expresión del *return* se coloca directamente al final del cuerpo del *defun* sin ninguna construcción adicional. Para capturar esto gramaticalmente, introducimos una nueva regla *cuerpo_funcion* que distingue si el cuerpo termina en *return* o no, y simplificamos *funcion* a una única alternativa que la usa:

```
C/C++
cuerpo_funcion:
    lista_sentencias RETURN expresion ';' { sprintf(temp, "%s\n%s",
$1.code, $3.code) ;
                                        $$code = gen_code(temp) ; }
    | RETURN expresion ';'
    | lista_sentencias
    ;
funcion:
    nombre_funcion '(' parametros_def ')' '{' declaraciones_locales lista_sentencias '}'
    { sprintf (temp, "(defun %s (%s)%s\n%s\n)", $1.code, $3.code,
    $6.code, $7.code) ;
    $$code = gen_code (temp) ; }
    ;
```

El segundo caso corresponde a un *return* en cualquier otro punto de la función, típicamente dentro de una estructura de control, como un *if* o un *while*. Para esto, hemos separado las sentencias del cuerpo principal de las sentencias que van dentro de las llaves de las estructuras de control. Para estas últimas, hemos creado la regla *lista_sentencias_bloque*, que permite combinar sentencias normales con retornos.

C/C++

```

lista_sentencias_bloque:
    sentencia                                { $$ = $1; }
    | RETURN expresion ';'                  { sprintf(temp, "(return-from %s %s)",
                                           funcion_actual, $2.code);
                                           $$code = gen_code(temp); }
    | lista_sentencias_bloque sentencia    { sprintf(temp, "%s\n%s",
                                           $1.code, $2.code);
                                           $$code = gen_code(temp); }
    | lista_sentencias_bloque RETURN expresion ';'
      { sprintf(temp, "%s\n(return-from %s %s)", $1.code,
        funcion_actual, $3.code);
        $$code = gen_code(temp); }
    ;

```

Finalmente, hemos actualizado las reglas de todas las estructuras de control (*if*, *while*, *for*, *switch*) para que consuman *lista_sentencias_bloque* en vez de la lista general.

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada: <pre> square (int v){ return (v*v); } fact (int n) { int f; if (n == 1) { f = 1; } else { f = n * fact(n-1); } return f; } abs_val (int x) { if (x < 0) { return -x; } return x; } main() { print("%d\n", square(7)); </pre>	Salida: <pre> (defun square (v) (return-from square (* v v))) (defun fact (n) (setq fact_f 0) (if (= n 1) (progn (setf fact_f 1)) (progn (setf fact_f (*n (fact (- n 1)))))) fact_f) (defun abs_val (x) (if (< x 0) (progn (return-from abs_val (-x)))) x) (defun main () (princ (square 7)) </pre>
---	---

<code>print("%d\n", fact(7));</code>	<code>(princ (fact 7))</code>
<code>print("%d\n", abs_val(-5));</code>	<code>(princ (abs_val (-5)))</code>
<code>print("%d\n", abs_val(5));</code>	<code>(princ (abs_val 5))</code>
<code>}</code>	<code>)</code>
<code>//@ (main)</code>	<code>(main)</code>

11.d. Llamadas a funciones como procedimientos

Este apartado quedó implementado de forma natural en la especificación 11.a), donde ya añadimos la producción *sentencia* que permite llamar a una función ignorando su valor de retorno:

C/C++

```
| IDENTIF '(' parametros_llamada ')' ';' { ... }
```

En Lisp, simplemente se llama a la función sin envolverla en ninguna construcción adicional, lo que equivale semánticamente a usarla como procedimiento.

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	Salida:
<code>square (int v){</code>	<code>(defun square (v)</code>
<code> return (v*v);</code>	<code>(return-from square (* v v))</code>
<code>}</code>	<code>)</code>
<code>main() {</code>	<code>(defun main ()</code>
<code> square(7);</code>	<code>(square 7)</code>
<code>}</code>	<code>)</code>
<code>//@ (main)</code>	<code>(main)</code>

FRONTEND - Especificación 12: Implementación de Vectores

Para implementar esta parte, hemos ido abordando el problema de forma incremental.

12.a. Declaración de vectores

Para soportar la declaración de vectores, extendemos las reglas *lista* y *lista_local* añadiendo dos producciones a cada una que reconocen la sintaxis *IDENTIF* *['* *NUMBER* *']* y la traducen a *(setq nombre (make-array tamaño))* . El tamaño se trata como *NUMBER* y no como *expresion* porque se especifica que será un valor entero.

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada: <pre>int b[5]; main (){ int v[3]; printf("s", b); } // @ (main)</pre>	Salida: <pre>(setq b (make-array 5)) (defun main () (setq main_v (make-array 3)) (princ b)) (main)</pre>
---	--

12.b. Acceso a elementos de un vector (operando)

Para acceder a un elemento de un vector dentro de una expresión, añadimos una producción a *operando* que reconoce *IDENTIF* `[' expresion ']` y genera la función *aref* de Lisp. El índice se trata como *expresion* para permitir tanto literales como operaciones aritméticas. Usamos *get_var_name* para resolver correctamente tanto vectores globales como locales.

PRUEBAS:

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada: <pre>int b[5]; main (){ printf("%s", b[0]); printf("s", b[2+1]); } // @ (main)</pre>	Salida: <pre>(setq b (make-array 5)) (defun main () (princ (aref b 0)) (princ (aref b (+ 2 1)))) (main)</pre>
--	---

12.c. Modificación de elementos de un vector (receptor)

Para modificar un elemento de un vector, añadimos una producción a *sentencia* que reconoce la asignación *IDENTIF* `[' expresion ']=' expresion ';' y genera (setf (aref nombre indice) valor). Tanto el índice como el valor se tratan como expresion para permitir operaciones arbitrarias en ambos lados. Usamos get_var_name para resolver correctamente vectores globales y locales.`

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada: <pre>int b[5]; main (){ b[0] = 123; b[2+1] = b[0] + 1; printf("%s", b[0]); printf("s", b[3]); } // @ (main)</pre>	Salida: <pre>(setq b (make-array 5)) (defun main () (setf (aref b 0) 123) (setf (aref b (+ 2 1)) (+ (aref b 0) 1)) (princ (aref b 0)) (princ (aref b 3))) (main)</pre>
---	--

12.d. Vectores como globales, locales y argumentos

En este apartado no hemos cambiado nada de código ya que la declaración de vectores tanto globales como locales quedó cubierta en el 12.a) . El paso de elementos indexados como argumentos en llamadas a funciones funciona de forma natural porque *lista_param_llamada* acepta *expresion*, e *IDENTIF* '[' *expresion* ']' ya reduce a *operando* y por tanto a *expresion* desde la 12.b). La restricción de no contemplar vectores como parámetros formales de una función tampoco requiere ninguna acción (simplemente no se añade esa producción a *lista_param_def* .

Para comprobar el correcto funcionamiento de esta parte, hemos realizado las siguientes pruebas:

- Entrada:	Salida:
<code>int v[5];</code>	<code>(setq b (make-array 5))</code>
<code>max (int x, int y) {</code>	<code>(defun max (x y)</code>
<code>if (x > y) {</code>	<code>(if (> x y)</code>
<code>return x;</code>	<code>(progn (return-from max x))</code>
<code>}</code>	<code>)</code>
<code>return y;</code>	<code>y</code>
<code>}</code>	<code>)</code>
<code>main (){</code>	<code>(defun main ()</code>
<code>v[0] = 3;</code>	<code>(setf (aref v 0) 3)</code>
<code>v[1] = 7;</code>	<code>(setf (aref v 1) 7)</code>
<code>printf("%s", max(v[0], v[1]));</code>	<code>(princ (max (aref v 0) (aref v 1)))</code>
<code>}</code>	<code>)</code>
<code>// @ (main)</code>	<code>(main)</code>

BACKEND - Especificación 1: Variables Globales

Para implementar la declaración de variables globales, añadimos los tokens *SETQ* y *SETF* a la tabla de palabras reservadas *keywords* [].

A nivel sintáctico, completamos la acción semántica de la producción ya existente. Si el valor inicial es 0, solo se declara la variable con *variable* *<var>* . Si tiene un valor distinto de 0, se declara y se inicializa con *variable* *<var>* *<var>* *<var>* ! .

C/C++

```
| '(' SETQ IDENTIF NUMBER ')' { if ($4.value == 0)
                                printf(" variable %s \n", $3.code);
                                else
                                printf(" variable %s %d %s ! \n",
                                    $3.code, $4.value, $3.code);
                                }
```

BACKEND - Especificación 2: Función MAIN

Para traducir la definición de la función principal, completamos la acción semántica de la producción *DEFUN MAIN* . Utilizamos acciones semánticas intermedias para imprimir : *main* antes de procesar el cuerpo y ; al cerrarlo.

C/C++

```
| '(' DEFUN MAIN { printf(" : main "); }
| '(' ')' exprSeq ')' { printf(" ; \n"); }
```

La llamada a *main* al final ya estaba implementada, mediante la producción *(' MAIN ')* que imprime *main* .

BACKEND - Especificación 3: Impresión de cadenas

Para traducir la impresión de cadenas, completamos la acción semántica de la producción *PRINT STRING*. En Forth la palabra *."* seguida de un espacio inicia la cadena y el *"* final la cierra. Añadimos también *cr* para el salto de línea, equivalente al comportamiento de *print* en Lisp.

C/C++

```
| '(' PRINT STRING ')' { printf(" .\" %s\" cr \n", $3.code); }
```

BACKEND - Especificación 4: Impresión de enteros

Para traducir la impresión de valores enteros y cadenas con *princ*, añadimos dos producciones separando el caso de expresión del caso de cadena. En Forth la palabra `.` imprime el valor entero de la cima de la pila, y `."` imprime una cadena sin salto de línea.

C/C++

```
| '(' PRINC expression ')'          { printf(" . \n"); }  
| '(' PRINC STRING ')'             { printf(" .\" %s\" \n", $3.code); }
```

BACKEND - Especificación 5: Asignación

Para traducir la asignación de un valor a una variable, completamos la acción semántica de la producción *SETF* ya presente en la plantilla. En Forth el operador `!` (store) toma de la pila un valor y una dirección de memoria y almacena el valor en dicha dirección.

C/C++

```
| '(' SETF IDENTIF expression ')'  { printf(" %s ! \n", $3.code); }
```

BACKEND - Especificación 6: Estructura de control WHILE

Para traducir el bucle `while`, usamos acciones semánticas intermedias que imprimen las palabras clave de Forth en el momento correcto.

C/C++

```
| '(' LOOP WHILE  
  expression          { printf(" begin \n"); }  
  DO exprSeq ')'      { printf(" while \n"); }  
                      { printf(" repeat \n"); }
```

BACKEND - Especificación 7: Estructura de control IF

La estructura *IF* ya estaba parcialmente implementada mediante *ifHead*. Completamos la traducción añadiendo el soporte para *PROGN*, que el frontend genera para agrupar sentencias dentro de cada rama y que el backend debe ignorar traduciendo solo su contenido.

C/C++

```
| '(' PROGN exprSeq ')' { ; }
```

BACKEND - Especificación 8: Operadores

Para soportar los operadores aritméticos, lógicos y relacionales necesarios, añadimos sus tokens correspondientes a la cabecera (*AND*, *OR*, *NOT*, *MOD*, *NEQ*, *LEQ*, *GEQ*), junto con sus entradas correspondientes en la tabla de *keywords*, y sus producciones en la regla *expression*.

Cabe destacar que no es necesaria la sección *%prec* (precedencia) en el backend ya que el LISP recibido del frontend ya tiene la precedencia resuelta mediante la estructura de paréntesis. Además, en Forth los valores Verdadero y Falso se representan con -1 y 0, a diferencia de C donde se usan 1 y 0.